

Key Design Decisions

The design models the consortium as four connected domains: people/access, resources/maintenance, experiments/applications, and billing/payment. This follows the original description, where researchers, institutions, facilities, equipment, staff, usage events, and funding sources all interact.

A major decision was to separate resource definitions from resource usage. Facilities and equipment have type/status tables, while actual use is represented through applications, allocations, and usage records. This keeps static information, such as facility capability or equipment type, separate from operational events such as scheduling, maintenance, and billing. Related “events” were also split into several concrete tables, including experiment phases, actual allocations, usage records, and maintenance records, rather than being stored in one generic event table. This makes the database easier to query for the web interface.

The ERD and final database use an assignment table to connect equipment to facilities over time. This was important because the project description says equipment can be movable or shared across facilities. In the implementation, reallocating equipment closes the current assignment by setting end_date and inserts a new assignment row, preserving history instead of overwriting location data. The current equipment location is therefore represented by the assignment row where end_date IS NULL.

Applications were designed as the gate between requests and real allocations. A researcher submits an application for either equipment or a facility, and actual allocation records are tied back to approved applications. The schema keeps the column name application_type, but allows it to represent either equipment or facility requests. It also stores concrete application fields such as applicant, experiment, requested dates, safety notes, estimated usage or outcome, selected equipment or facility, and supervising staff. The implemented schema uses triggers to prevent actual allocations unless the application is approved.

For billing, the design stores charge status directly in the charge table instead of deriving it only from payment records. This supports simple filtering for statuses such as unpaid, paid, and disputed, and makes the billing view easier to implement in the UI.

The UI was implemented using plain Flask HTML templates with SQLite as the backend database. This avoided extra framework complexity while still supporting searchable resource lists, application forms, scheduling views, billing views, and dashboard statistics. The interface keeps a simple multi-section layout, allowing users to move between functional views through standard page and template navigation without relying on a full front-end framework. The database connection helper enables SQLite foreign keys, which support referential integrity in the implemented database.

Assumptions Made

The original description is broad and business-oriented, so several assumptions were needed when translating it into an ERD.

First, each person was assumed to have one primary occupation, either researcher or technical staff. The original text describes different staff and researcher roles, but does not define a full role hierarchy.

Second, applications were assumed to request either equipment or a facility, not an unlimited bundle of multiple facilities and multiple equipment items. The original text says an experiment may involve multiple resources, but the ERD simplifies this by allowing each application to focus on one selected equipment item or one selected facility.

Third, each application is assumed to belong to one experiment. This makes the application process traceable from request to experiment to allocation and usage. The implemented API enforces this when creating applications.

Fourth, technical staff support was simplified to one supervising staff_id on each application. The original text allows multiple staff members and changing assignments over time, but the model keeps this as a single staff reference for clarity.

Fifth, billing responsibility was modeled through `charge.person_id`, `funding_source`, `charge_allocation`, and `payment`. This assumes that each charge has one responsible person but may be paid or allocated across funding sources.

Tradeoffs and Limitations

The main tradeoff is that the model is practical and demo-friendly, but less flexible than the real consortium described in the brief.

The biggest limitation is resource scope. The original text allows one experiment to involve multiple facilities, multiple pieces of equipment, staff teams, setup periods, and changing needs over time. The implemented application model supports either an equipment request or a facility request at a time. This is simpler for forms and queries, but it cannot fully represent a complex multi-resource experiment in one application.

Staff tracking is also simplified. The system records staff on applications and maintenance activities, but it does not model staff schedules, expertise areas, staff availability conflicts, or multiple staff members supporting one experiment.

Access control is represented through credentials and usage records, but role-based visibility and confidentiality controls are not fully implemented. The original brief mentions sensitive data and different visibility by role, while the UI has no login/authentication layer.

Scheduling is date-based rather than capacity- or conflict-based. The UI shows actual allocations in calendar/list views, but it does not automatically detect conflicts, propose substitute resources, or optimize fairness across institutions.

Billing is functional but simplified. The system supports charges, adjustments, allocations, payments, and budget summaries, but it does not model recurring billing cycles, provisional versus finalized charges, complex funding restrictions, or changing allocation rules over time. The payment action simply creates internal-transfer payment rows from existing charge allocations and marks the charge paid.

Handling Ambiguities

When the description was ambiguous, the design generally chose a normalized but simple interpretation.

For facility purposes, the phrase “facilities can serve multiple purposes over time” was interpreted by creating a purpose lookup and assigning each facility a current purpose, rather than modeling a full history of facility reconfiguration.

For movable equipment, the design uses an assignment table. Instead of storing `facility_id` directly on equipment only, the final database tracks equipment-to-facility assignments with start and end dates.

For resource requests, the broad description of “facility type, equipment requirements, duration, staffing needs, safety considerations, and special conditions” was converted into concrete application fields: applicant, experiment, application type, requested dates, estimated usage, selected equipment/facility, and staff.

For events, the design separates the concept into several specific tables rather than one generic event table: experiment phases, actual allocations, usage records, and maintenance records. This makes queries easier for the UI, but it reduces flexibility compared with a universal event model.

For billing, charge status is stored directly instead of being derived from payments. This allows statuses such as unpaid, paid, and disputed, which supports simpler billing filtering in the UI.

Distribution of the teamwork

We would like to acknowledge the contributions of each team member throughout the project. For Milestone 1, all four team members worked together on the ERD, with each person responsible for a specific section. The draft analysis report was also completed collaboratively, summarizing the work done for each part of the ERD.

For Milestone 2, Myra Jiang and Yiduo Lu worked on the proposed queries and website mockup, while Yuhong Geng and Jingya Zhang completed the initial database export.

For the final project, Myra Jiang, Yiduo Lu, and Jingya Zhang developed the web interface, and Yuhong Geng prepared the presentation slides.